# Data Science — laboratorium

## NumPy: obsługa tablic i wektorów — część I

### Cel zajęć

Na tych zajęciach poznamy podstawowy obiekt biblioteki NumPy — **tablicę `ndarray`**. Nauczymy się tworzyć tablice jedno- i dwuwymiarowe, sprawdzać ich rozmiar i typ danych, wykonywać obliczenia na całych tablicach oraz wybierać ich elementy i fragmenty.

NumPy jest podstawowym narzędziem obliczeń numerycznych w Pythonie. Jedną z jego najważniejszych cech jest możliwość wykonywania działań na całych tablicach bez konieczności samodzielnego pisania pętli przetwarzających każdy element

Każdy przykład wpisuj samodzielnie do osobnej komórki **Google Colab** i uruchamiaj klawiszami **Shift + Enter**.

# 1. Zaczynamy — import NumPy

Bibliotekę NumPy standardowo importujemy pod skróconą nazwą `np`:

```
import numpy as np
```

Sprawdźmy, czy działa:

```
print(np.__version__)
```

### Obserwacja

Od tej chwili funkcje biblioteki wywołujemy przez:

```
np.nazwa_funkcji()
```

na przykład:

```
np.array(...)
```

### Małe zadanie

Uruchom:

```
import numpy as np
print(np.pi)
print(np.sqrt(16))
```

Co zwracają oba wyrażenia?

# 2. Od listy Pythona do tablicy NumPy

Zwykła lista:

```
data = [10, 20, 30, 40]
print(data)
print(type(data))
```

Teraz utwórzmy na jej podstawie tablicę NumPy:

```
arr = np.array(data)
print(arr)
print(type(arr))
```

### Obserwacja

`arr` nie jest listą.

Jej typ to:

```
numpy.ndarray
```

`ndarray` jest podstawowym wielowymiarowym obiektem tablicowym NumPy.

### Małe zadanie

Utwórz tablicę:

```
temperatures = np.array([18, 21, 19, 23, 25])
```

Wyświetl:

```
temperatures
```

a następnie:

```
type(temperatures)
```

# 3. Tablica jednowymiarowa — wektor

Utwórz:

```
v = np.array([2, 4, 6, 8, 10])
```

Sprawdź trzy właściwości:

```
print(v.ndim)
print(v.shape)
print(v.dtype)
```

### Obserwacja

`ndim` — liczba wymiarów tablicy,

`shape` — rozmiar tablicy w każdym wymiarze,

`dtype` — typ przechowywanych danych.

McKinney wyróżnia właśnie `ndim`, `shape` i `dtype` jako podstawowe informacje opisujące tablicę `ndarray`.

### Małe zadanie

Utwórz:

```
x = np.array([3.5, 7.2, 1.8, 9.0])
```

Przewidź, a dopiero potem sprawdź:

```
x.ndim
x.shape
x.dtype
```

Czy `dtype` jest taki sam jak w poprzedniej tablicy?

# 4. Tablica dwuwymiarowa

NumPy nie ogranicza się do wektorów.

```
data = np.array([[10, 20, 30],[40, 50, 60]])
print(data)
```

Teraz:

```
print(data.ndim)
print(data.shape)
```

### Obserwacja

Tablica ma dwa wymiary.

```
data.shape
```

zwraca:

```
(2, 3)
```

czyli:
**dwa wiersze × trzy kolumny**.

### Małe zadanie

Utwórz tablicę:

```
grades = np.array([ [4, 5, 3], [5, 4, 4], [3, 3, 5], [5, 5, 4]])
```

Bez uruchamiania przewidź:

```
grades.ndim
grades.shape
```

Dopiero potem sprawdź odpowiedź w Colabie.

# 5. Tworzenie tablic bez ręcznego wpisywania danych

NumPy posiada funkcje pozwalające szybko tworzyć całe tablice. McKinney omawia m.in. `zeros`, `ones`, `empty` i `arange`.

### Tablica zer

```
a = np.zeros(5)
print(a)
```

Tablica dwuwymiarowa:

```
b = np.zeros((3, 4))
print(b)
```

### Tablica jedynek

```
c = np.ones((2, 3))
print(c)
```

### Sekwencja liczb

```
d = np.arange(10)
print(d)
```

Spróbuj:

```
np.arange(5, 15)
```

oraz:

```
np.arange(0, 20, 2)
```

### Małe zadanie

Utwórz bez ręcznego wpisywania wszystkich elementów:

1. tablicę sześciu zer,
2. tablicę `3 × 3` wypełnioną jedynkami,
3. liczby od `0` do `20`,
4. liczby parzyste od `10` do `20`.

# 6. Typ danych — `dtype`

Utwórz:

```
a = np.array([1, 2, 3, 4])
print(a.dtype)
```

Następnie:

```
b = np.array([1.0, 2.0, 3.0, 4.0])
print(b.dtype)
```

A teraz:

```
c = np.array([1, 2.5, 3])
print(c)
print(c.dtype)
```

### Obserwacja

Elementy jednej tablicy NumPy mają wspólny typ danych.

Jeżeli połączymy liczby całkowite i zmiennoprzecinkowe, NumPy dobierze typ pozwalający przechować wszystkie wartości.

NumPy udostępnia własne precyzyjnie określone typy numeryczne, m.in. różne warianty liczb całkowitych i zmiennoprzecinkowych.

# 7. Zmiana typu — `astype()`

Utwórz:

```
a = np.array([1, 2, 3, 4])
print(a)
print(a.dtype)
```

Teraz:

```
b = a.astype(float)
print(b)
print(b.dtype)
```

Spróbuj odwrotnie:

```
x = np.array([2.8, 4.1, 7.9])
y = x.astype(int)
print(y)
print(y.dtaype)
```

### Obserwacja

Zwróć uwagę na wynik.

Czy liczby zostały **zaokrąglone**, czy została po prostu odrzucona część ułamkowa?

`astype()` jest w NumPy podstawową metodą jawnej konwersji typu tablicy.

### Małe zadanie

Przewidź wynik:

```
x = np.array([1.9, 2.1, 3.7, 4.99])
print(x.astype(int))
```

Sprawdź swoją odpowiedź.

# 8. Najważniejsza różnica — działania na całej tablicy

Utwórz:

```
x = np.array([1, 2, 3, 4, 5])
```

Teraz wykonaj:

```
print(x + 10)
print(x * 2)
print(x ** 2)
print(x / 2)
```

### Obserwacja

Nie napisaliśmy żadnej pętli `for`.

Operacja została wykonana na **każdym elemencie tablicy**.

Takie wykonywanie operacji na całych tablicach jest jedną z kluczowych idei NumPy i podstawą późniejszej pracy z danymi.

# 9. Operacje dla dwóch tablic

Utwórz:

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
```

Sprawdź:

```
print(a + b)
print(a * b)
print(b - a)
```

### Pytanie

Co oznacza:

```
a * b
```

Czy jest to mnożenie macierzy?

Sprawdź wynik i zastanów się, jaka operacja została wykonana.

### Mnożenie macierzy — operator `@`

Utwórz dwie tablice `2 × 2`:

```
a = np.array([[1, 2],[3, 4]])
b = np.array([[5, 6],[7, 8]])
```

Najpierw sprawdź:

```
print(a * b)
```

Otrzymamy:

```
[[ 5 12]
 [21 32]]
```

Operator `*` wykonuje **mnożenie element po elemencie**.

Teraz wykonaj:

```
print(a @ b)
```

Wynik:

```
[[19 22]
 [43 50]]
```

Operator `@` wykonuje **mnożenie macierzy**.

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

mnożenie macierzy $A \cdot B$ liczymy **wiersz × kolumna**:

$$A@B = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix}$$

Czyli:

$$= \begin{bmatrix} 5 + 14 & 6 + 16 \\ 15 + 28 & 18 + 32 \end{bmatrix} = \boxed{\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}}$$

## Małe zadanie

Porównaj:

```
print(a * b)
print(a @ b)
```

**Pytanie:** Dlaczego wyniki są różne?

Zapamiętaj:

```
a * b    → mnożenie element po elemencie (element-wise multiplication)
a @ b    → mnożenie macierzy
```

### Małe zadanie

Dane są:

```
prices = np.array([10, 15, 20, 25])
```

Podnieś wszystkie ceny:

**a)** o `5` zł,

**b)** dwukrotnie,

**c)** o dziesięć procent.

Spróbuj wykonać każde zadanie jednym wyrażeniem NumPy.

# 10. Indeksowanie

Indeksowanie działa podobnie jak w listach Pythona.

```
a = np.array([10, 20, 30, 40, 50])
```

Sprawdź:

```
print(a[0])
print(a[2])
print(a[-1])
```

Możemy także zmienić element:

```
a[2] = 999
print(a)
```

# 11. Wycinanie — slicing

Utwórz:

```
a = np.arange(10)
print(a)
```

Teraz:

```
print(a[2:6])
print(a[:4])
print(a[5:])
```

### Małe zadanie

Dla:

```
a = np.arange(20)
```

wyświetl:

- pierwsze pięć elementów,
- elementy od indeksu `5` do `9`,
- ostatnie pięć elementów,
- co drugi element.

Podstawowe indeksowanie i wycinanie tablic jest kolejnym etapem pracy z `ndarray` omawianym przez McKinneya.

# 12. Uwaga: wycinek może być widokiem

To zachowanie jest bardzo ważne.

```
a = np.array([10, 20, 30, 40, 50])
fragment = a[1:4]
print(fragment)
```

Teraz:

```
fragment[0] = 999
print(fragment)
print(a)
```

### Obserwacja

Zmieniła się również tablica `a`.

W NumPy podstawowy wycinek tablicy zwykle jest **widokiem na dane**, a nie niezależną kopią. McKinney specjalnie zwraca uwagę na tę różnicę względem zwykłych list Pythona.

Jeżeli potrzebujemy kopii:

```
fragment = a[1:4].copy()
```

### Małe zadanie

Powtórz poprzedni eksperyment używając `.copy()`.

Czy zmiana `fragment` nadal modyfikuje `a`?

# 13. Indeksowanie tablicy dwuwymiarowej

Utwórz:

```
data = np.array([[10, 20, 30],[40, 50, 60],[70, 80, 90]])
```

Element w drugim wierszu i trzeciej kolumnie:

```
print(data[1, 2])
```

Cały pierwszy wiersz:

```
print(data[0])
```

Fragment tablicy:

```
print(data[:2, :2])
```

Schemat indeksowania tablic dwuwymiarowych pokazany w książce dobrze ilustruje, że kolejne indeksy odpowiadają kolejnym osiom tablicy.

### Małe zadanie

Dla tablicy `data` wyświetl:

- trzeci wiersz,
- drugą kolumnę,
- wartość `80`,
- lewy górny fragment `2 × 2`.

# 14. Mini-zadanie Data Science

Załóżmy, że mamy pomiary temperatury z kolejnych dni:

```
temperatures = np.array([18.5, 20.1, 21.0, 19.8, 23.4, 25.0, 24.1])
```

Wykonaj kolejno:

```
print(temperatures.shape)
print(temperatures.ndim)
print(temperatures.dtype)
```

Następnie utwórz tablicę temperatur podwyższonych o dwa stopnie:

```
warmer = temperatures + 2
```

Wyświetl pierwsze trzy pomiary z `warmer`:

```
print(warmer[:3])
```

Na koniec zamień temperatury na liczby całkowite:

```
print(temperatures.astype(int))
```

### Pytania

1. Ile wymiarów ma ta tablica?
2. Ile zawiera obserwacji?
3. Czy dodanie `2` wymagało pętli?
4. Czy `astype(int)` zmieniło oryginalną tablicę?
5. Jaki jest typ danych tablicy `warmer`?

# Na koniec — sprawdź, czy już umiesz

Po pierwszej części powinieneś potrafić wyjaśnić:

- czym różni się lista Pythona od tablicy `ndarray`,
- do czego służą `ndim`, `shape` i `dtype`,
- jak działają `np.array()`, `np.zeros()`, `np.ones()` i `np.arange()`,
- jak zmieniać typ danych za pomocą `astype()`,
- jak wykonywać działania na całej tablicy bez pętli,
- jak wybierać pojedyncze elementy i fragmenty tablic,
- dlaczego wycinek NumPy nie zawsze jest niezależną kopią danych.